

Problem Set 2

Fundamentals of Numerical Programming I

LEONARD STORCKS

January 4, 2026

Problem 1. Tridiagonal solver

In this task we want to practice the principle of Gaussian elimination on a simplified setup. Consider the invertible tridiagonal matrix

$$\underline{\underline{A}} = \begin{pmatrix} d_0 & u_0 & 0 & \dots & 0 & 0 \\ l_1 & d_1 & u_1 & 0 & \dots & \\ 0 & l_2 & d_2 & u_2 & 0 & \dots \\ 0 & 0 & \ddots & \ddots & \ddots & 0 \\ 0 & \dots & & l_{N-2} & d_{N-2} & u_{N-2} \\ 0 & 0 & \dots & 0 & l_{N-1} & d_{N-1} \end{pmatrix} \quad (1)$$

(by convention $l_0 = 0, u_{N-1} = 0$) where we want to solve

$$\underline{\underline{A}}\underline{x} = \underline{b} \quad (2)$$

for $\underline{x}$.

- Write the system in the form $(\underline{\underline{A}}|\underline{b})$. Find the row operations which eliminate the lower diagonal, starting with l_1 . *Hint*: You start by subtracting a multiple of the first row from the second such that $l_1 = 0$ and continue downward.
- Once the lower diagonal is eliminated, the system can be solved bottom-up. Given the transformed system with eliminated lower diagonal, find an expression for the solution vector. *Hint*: Start by finding an expression for the last entry x_{N-1} in $\underline{x} = (x_0, x_1, \dots, x_{N-2}, x_{N-1})^T$, continue with x_{N-2} and so on.
- Implement the tridiagonal solver in Python. The signature of the function as well as the implementation of a matrix-vector product are given in Code-Snippets 1 to 3. What

```

1      """
2      The tridiagonal matrix is given by
3      [u_0, u_{N-2}], [d_0, d_{N-1}], [l_1, l_{N-1}].
4      By convention all arrays are of size N, with
5      u_{N-1} = 0, l_0 = 0. This way the
6      (l_i, d_i, u_i) are the values
7      from the i-th row.
8      """
9
10     import numpy as np
11     import matplotlib.pyplot as plt
12     from scipy.linalg import lu_factor, lu_solve
13
14     def tri_vector_product(l, d, u, x):
15         """
16         Compute Ax where A is a tridiagonal
17         matrix defined by l, d, u.
18         """
19         return x * d + np.roll(x, 1) * l + np.roll(x, -1) * u
20
21     def tri_to_matrix(l, d, u):
22         """
23         Convert a tridiagonal matrix to a
24         full matrix.
25         """
26         N = d.shape[0]
27         return np.diag(d) + np.diag(l[1:], -1) + np.diag(u[:-1], 1)

```

Code-Snippet 1: Python code for a matrix-vector product $\underline{\underline{A}}\underline{\underline{x}}$ where $\underline{\underline{A}}$ is tridiagonal and signature for a tridiagonal solve of $\underline{\underline{A}}\underline{\underline{x}} = \underline{\underline{b}}$. Continued in Code-Snippet 2.

is the scaling of your algorithm? Compare to $\mathcal{O}(N^3)$ of standard LU factorization.

```

1      def tri_solve(l, d, u, b):
2          """
3              Solve  $Ax = b$  where  $A$  is a tridiagonal
4              matrix defined by  $l$ ,  $d$ ,  $u$ .
5
6              Args:
7                   $l$ ,  $d$ ,  $u$ : arrays of size  $N$ 
8                   $b$ : array of size  $N$ 
9
10             Returns:
11                 Solution to  $Ax = b$  of size  $N$ 
12             """
13             N = diag.shape[0]
14
15             # backward elimination of
16             # the lower diagonal
17             for i in range(0, N - 1):
18                 # TODO
19
20             # forward substitution
21             x = np.zeros_like(diag)
22             x[-1] = # TODO
23
24             for i in range(N - 2, -1, -1):
25                 # TODO
26
27             return x

```

Code-Snippet 2: Continuation of Code-Snippet 1. Continued in Code-Snippet 3

```

1      # test setup
2      l = np.array([0.0, 2.0, 4.0, 6.0, 2.0, 6.0])
3      d = np.array([7.0, 9.0, 10.0, 15.0, 9.0, 7.0])
4      u = np.array([4.0, 6.0, 5.0, 8.0, 7.0, 0.0])
5
6      # predefined solution
7      x = np.array([5.0, 7.0, 8.0, 9.0, 5.0, 1.0])
8
9      # tridiagonal representation to full matrix
10     A = tri_to_matrix(l, d, u)
11
12     # matrix vector product
13     assert np.allclose(A @ x, tri_vector_product(l, d, u, x))
14     b = tri_vector_product(l, d, u, x)
15
16     # tridiagonal solve
17     x_tilde = tri_solve(l, d, u, b)
18     assert np.allclose(x, x_tilde)

```

Code-Snippet 3: Continuation of Code-Snippet 2.

Solution

Part (a)

To eliminate the lower diagonal l_i , we proceed from $i = 1$ to $N - 1$. For each row i , we subtract a multiple of the modified row $i - 1$ from row i . Let the modified diagonal elements be d'_i and the modified right-hand side elements be b'_i .

(a) For the first row ($i = 0$), the elements remain unchanged:

$$d'_0 = d_0, \quad b'_0 = b_0 \quad (3)$$

(b) For $i = 1, \dots, N - 1$, we calculate the multiplier $m_i = \frac{l_i}{d'_{i-1}}$. The row operations are:

- $d'_i = d_i - m_i u_{i-1}$
- $b'_i = b_i - m_i b'_{i-1}$

The upper diagonal u_i remains unchanged because the element above it in the previous row's column is zero. After these operations, the matrix is upper bidiagonal.

Part (b)

After the lower diagonal is eliminated, the system is in upper triangular form:

$$\left(\begin{array}{cccc|c} d'_0 & u_0 & & & b'_0 \\ & d'_1 & u_1 & & b'_1 \\ & & \ddots & \ddots & \vdots \\ & & & d'_{N-2} & u_{N-2} & b'_{N-2} \\ & & & & d'_{N-1} & b'_{N-1} \end{array} \right) \quad (4)$$

We solve this using back-substitution starting from the last row:

- (a) For the last entry: $x_{N-1} = \frac{b'_{N-1}}{d'_{N-1}}$
- (b) For $i = N - 2, N - 3, \dots, 0$:

$$d'_i x_i + u_i x_{i+1} = b'_i \implies x_i = \frac{b'_i - u_i x_{i+1}}{d'_i} \quad (5)$$

Part (c)

The implementation of the Thomas algorithm (tridiagonal matrix solver) is given in Code-Snippet 4.

```

1      def tri_solve(l, d, u, b):
2          N = d.shape[0]
3          # Copy to avoid modifying original arrays
4          d_copy = d.astype(float).copy()
5          b_copy = b.astype(float).copy()
6
7          # backward elimination of the lower diagonal (forward sweep)
8          for i in range(0, N - 1):
9              m = l[i+1] / d_copy[i]
10             d_copy[i+1] -= m * u[i]
11             b_copy[i+1] -= m * b_copy[i]
12
13             # forward substitution (back substitution sweep)
14             x = np.zeros_like(d_copy)
15             x[-1] = b_copy[-1] / d_copy[-1]
16
17             for i in range(N - 2, -1, -1):
18                 x[i] = (b_copy[i] - u[i] * x[i+1]) / d_copy[i]
19
20         return x

```

Code-Snippet 4: Thomas algorithm (tridiagonal matrix solver).

Scaling Analysis: The algorithm consists of two loops, each iterating N times. Inside each loop, only a constant number of floating-point operations (additions, multiplications, divisions) are performed. Therefore, the complexity is $\mathcal{O}(N)$.

Compared to the standard Gaussian elimination or LU factorization which scales as $\mathcal{O}(N^3)$, this solver is significantly more efficient for large tridiagonal systems, reducing both computation time and memory requirements (since we only store vectors of size N rather than a matrix of size N^2).

Problem 2. Jacobi iteration

Consider random strictly diagonally dominant matrices $\underline{\underline{R}}_N$ with

$$\forall i : |r_{ii}| > \sum_{j \neq i} |a_{ij}| \quad (6)$$

These can be generated by

- (a) Draw a random Gaussian matrix $\underline{\underline{G}}_N \in \mathbb{R}^{n \times n}$, $G_{ij} \sim \mathcal{N}(\mu = 0, \sigma^2 = 1)$.
- (b) Enforce strict diagonal dominance by

$$R_{ij} = \begin{cases} G_{ij}, & i \neq j, \\ \delta + \sum_{j \neq i} |G_{ij}|, & i = j \end{cases} \quad (7)$$

with margin $\delta = 0.1$.

A Gaussian RHS in $\underline{\underline{A}}\underline{x} = \underline{b}$, $b_i \sim \mathcal{N}(\mu = 0, \sigma^2 = 1)$, $i = 1, \dots, N$ is used throughout this task.

Jacobi iteration: For Jacobi iteration, we start with the trivial decomposition

$$\underline{\underline{A}} = \underline{\underline{D}} - (\underline{\underline{L}} + \underline{\underline{U}}), \quad \text{diagonal part } \underline{\underline{D}}, \quad \text{negative left-below-diagonal part } \underline{\underline{L}} \\ \text{negative right-above-diagonal part } \underline{\underline{U}} \quad (8)$$

and iterate

$$\underline{x}^{(n+1)} = \underline{\underline{D}}^{-1}\underline{b} + \underbrace{\underline{\underline{D}}^{-1}(\underline{\underline{L}} + \underline{\underline{U}})}_{=: \underline{\underline{M}}} \underline{x}^{(n)}, \quad (\underline{\underline{D}}^{-1})_{ii} = \frac{1}{A_{ii}} \quad (9)$$

You can start the iteration with $\underline{x}^{(0)} = \underline{0}$.

- (a) Show that Jacobi iteration converges for a strictly diagonally dominant matrix, i.e. the spectral radius $\rho_s(\underline{\underline{M}}) < 1$.
- (b) Implement the generation of $\underline{\underline{R}}_N$ and $\underline{b} \in \mathbb{R}^N$ with $b_i \sim \mathcal{N}(\mu = 0, \sigma^2 = 1)$, $i = 1, \dots, N$ in Python.
- (c) Implement the Jacobi iteration in Python. For $N = 5, 50, 150$ plot the *normwise backward error*

$$\eta^{(n)} = \frac{\|\underline{r}^{(n)}\|_1}{\|\underline{\underline{A}}\|_1 \|\underline{\hat{x}}^{(n)}\|_1 + \|\underline{b}\|_1}, \quad \underline{r} = \underline{\underline{A}}\underline{\hat{x}}^{(n)} - \underline{b} \quad (10)$$

for $n = 0, 1, \dots, N$.

Solution

Part (a)

The Jacobi iteration matrix is $\underline{\underline{M}} = \underline{\underline{D}}^{-1}(\underline{\underline{L}} + \underline{\underline{U}})$. The components of $\underline{\underline{M}}$ are $M_{ij} = -A_{ij}/A_{ii}$ for $i \neq j$ and $M_{ii} = 0$. Convergence is guaranteed if the spectral radius $\rho_s(\underline{\underline{M}}) < 1$. Using the induced ∞ -norm (maximum row sum):

$$\|\underline{\underline{M}}\|_{\infty} = \max_i \sum_{j=1}^N |M_{ij}| = \max_i \sum_{j \neq i} \frac{|A_{ij}|}{|A_{ii}|} \quad (11)$$

Since $\underline{\underline{A}}$ is strictly diagonally dominant, $|A_{ii}| > \sum_{j \neq i} |A_{ij}|$ for all i . This implies $\frac{\sum_{j \neq i} |A_{ij}|}{|A_{ii}|} < 1$, so $\|\underline{\underline{M}}\|_{\infty} < 1$. Since $\rho_s(\underline{\underline{M}}) \leq \|\underline{\underline{M}}\|_{\infty}$, the iteration must converge.

Part (b)

The system generation is given in Code-Snipped 5.

```
1  def generate_system(N, delta=0.1):
2      G = np.random.normal(0, 1, (N, N))
3      R = G.copy()
4      for i in range(N):
5          # Sum of absolute values of non-diagonal elements
6          off_diag_sum = np.sum(np.abs(G[i, :])) - np.abs(G[i, i])
7          R[i, i] = delta + off_diag_sum
8      b = np.random.normal(0, 1, N)
9      return R, b
```

Code-Snippet 5: Generation of strictly diagonally dominant matrix $\underline{\underline{R}}_N$.

Part (c)

The Jacobi solver is given in Code-Snippet 6.

```
1  def jacobi_solve(A, b, max_iter):
2      N = len(b)
3      x = np.zeros(N)
4      D = np.diag(A)
5      LU = np.diag(D) - A # Matrix (L+U)
6
7      norm_A = np.linalg.norm(A, 1)
8      norm_b = np.linalg.norm(b, 1)
9      errors = []
10
11     for n in range(max_iter + 1):
12         # Calculate normwise backward error
13         r = A @ x - b
14         eta = np.linalg.norm(r, 1) / (norm_A * np.linalg.norm(x, 1)
15         ↪ + norm_b)
16         errors.append(eta)
17
18         # Jacobi Update
19         if n < max_iter:
20             x = (b + LU @ x) / D
21     return errors
22
23     # Plotting logic
24     for N in [5, 50, 150]:
25         A, b = generate_system(N)
26         errs = jacobi_solve(A, b, N)
27         plt.semilogy(range(N + 1), errs, label=f'N={N}')
```

Code-Snippet 6: Jacobi implementation and error tracking.

Problem 3. Floating point errors in linear solves

Floating point precision is limited such that even for an exact method such as LU factorization our result for $\underline{x}$ in $\underline{A}\underline{x} = \underline{b}$ will only be an approximation $\hat{\underline{x}}$.

A typical error measure for linear solvers is the *normwise backward error*

$$\eta = \frac{\|\underline{r}\|_1}{\|\underline{A}\|_1 \|\hat{\underline{x}}\|_1 + \|\underline{b}\|_1}, \quad \underline{r} = \underline{A}\hat{\underline{x}} - \underline{b} \quad (12)$$

For our tridiagonal solver from Problem 1, η will typically be around machine precision ϵ_{mach} , see Higham, 2002, Theorem 9.14.

In this task we will work with LU factorization with partial pivoting as one would use in practice, a minimal example is given in Code-Snippet 7.

We will consider two kinds of example matrices:

- random strictly diagonally dominant matrices $\underline{R}_N$ with

$$\forall i : |r_{ii}| > \sum_{j \neq i} |a_{ij}| \quad (13)$$

These can be generated by

```
1      import numpy as np
2      from scipy.linalg import lu_factor, lu_solve
3
4      # Define a small linear system A x = b
5      A = np.array([[3.0, 1.0],
6                    [1.0, 2.0]])
7      b = np.array([9.0, 8.0])
8
9      # LU factorization with partial pivoting
10     lu, piv = lu_factor(A)
11
12     # Solve the system using the LU factors
13     x = lu_solve((lu, piv), b)
14
15     print("Solution x:", x)
16
17     # Optional: check the result
18     print("A @ x:", A @ x)
```

Code-Snippet 7: Basic LU solve in Python.

- (a) Draw a random Gaussian matrix $\underline{G}_N \in \mathbb{R}^{n \times n}$, $G_{ij} \sim \mathcal{N}(\mu = 0, \sigma^2 = 1)$.
(b) Enforce strict diagonal dominance by

$$R_{ij} = \begin{cases} G_{ij}, & i \neq j, \\ \delta + \sum_{j \neq i} |G_{ij}|, & i = j \end{cases} \quad (14)$$

with margin $\delta = 0.1$.

- Wilkinson matrices

$$W_n = \begin{bmatrix} 1 & 0 & 0 & \cdots & 0 & 1 \\ -1 & 1 & 0 & \cdots & 0 & 1 \\ -1 & -1 & 1 & \ddots & \vdots & \vdots \\ \vdots & \vdots & \ddots & \ddots & 0 & 1 \\ -1 & -1 & \cdots & -1 & 1 & 1 \\ -1 & -1 & \cdots & -1 & -1 & 1 \end{bmatrix} \in \mathbb{R}^{n \times n}, \quad W_{ij} = \begin{cases} 1 & \text{if } i = j \text{ or } j = n \\ -1 & \text{if } i > j \\ 0 & \text{otherwise} \end{cases} \quad (15)$$

A Gaussian RHS $b_i \sim \mathcal{N}(\mu = 0, \sigma^2 = 1)$, $i = 1, \dots, N$ is used throughout this task.

- (a) Implement functions to generate $\underline{W}_N$ and $\underline{R}_N$.
(b) Implement a function to calculate η .
(c) For both $\underline{W}_N$ and $\underline{R}_N$ for $N = 5, 50, 100, 150$ calculate η . Plot η over N .
(d) For $\underline{W}_4$ go through Gaussian elimination with partial pivoting by hand. Explain the immense accumulation of floating point errors with increasing N .

Solution

Part (a)

TODO

Part (b)

TODO

Part (c)

TODO

Problem 4. (*Bonus*) Polynomial regression

Given the data

$$\begin{aligned} \underline{x} &= \begin{pmatrix} -3.00, -2.85, -2.70, -2.55, -2.40, -2.25, -2.10, -1.95, \\ -1.80, -1.65, -1.50, -1.35, -1.20, -1.05, -0.90, -0.75, \\ -0.60, -0.45, -0.30, -0.15, 0.00, 0.15, 0.30, 0.45, \\ 0.60, 0.75, 0.90, 1.05, 1.20, 1.35, 1.50, 1.65, \\ 1.80, 1.95, 2.10, 2.25, 2.40, 2.55, 2.70, 2.85, \end{pmatrix} \in \mathbb{R}^{40}, \\ \underline{y}^{\text{data}} &= \begin{pmatrix} -6.74, -6.01, -4.83, -4.37, -4.01, -2.94, -1.90, -1.56, \\ -1.92, -1.77, -1.07, -0.41, -1.30, 0.01, -0.28, 0.16 \\ 0.41, 0.64, 1.10, 1.48, 0.94, 1.70, 0.69, 1.17, \\ 1.41, 0.96, 0.48, 0.31, 0.46, 0.71, 0.01, 0.31, \\ 0.24, 0.50, 0.25, 0.24, -0.34, -0.15, 0.26, 0.57, \end{pmatrix} \in \mathbb{R}^{40} \end{aligned} \quad (16)$$

we would like to find parameters $\underline{\beta} \in \mathbb{R}^p$ in the relation

$$y_i(\underline{\beta}) = \beta_0 + \beta_1 x_i + \beta_2 x_i^2 + \dots + \beta_{p-1} x_i^{p-1} \quad (17)$$

such that $\|\underline{y}^{\text{data}} - \underline{y}(\underline{\beta})\|_2^2$ is minimized. Note that $y_i(\underline{\beta})$ is linear in $\underline{\beta}$, polynomial regression is a linear regression.

Given the data matrix (here $N = 40$)

$$\underline{\underline{X}} = \begin{bmatrix} 1 & x_1 & x_1^2 & \cdots & x_1^{p-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_N & x_N^2 & \cdots & x_N^{p-1} \end{bmatrix} \in \mathbb{R}^{N \times p} \quad (18)$$

The parameters of interest are given by

$$\hat{\underline{\beta}} = \underset{\underline{\beta}}{\operatorname{argmin}} \|\underline{y}^{\text{data}} - \underline{y}(\underline{\beta})\|_2^2 \iff \underline{\underline{X}}^T \underline{\underline{X}} \hat{\underline{\beta}} = \underline{\underline{X}}^T \underline{y}^{\text{data}} \quad (19)$$

i.e. a linear system.

- Show that $\underline{\underline{X}}^T \underline{\underline{X}}$ is invertible for $p < N$.
- Write Python functions to construct the data matrix $\underline{\underline{X}}$ and to evaluate $\underline{y}(\underline{\beta})$.
- Solve for $\underline{\beta}$ for $p = 1, 2, 4, 12$. Plot the results and the data. Describe what you observe.

Solution

Part (a)

TODO

References

Higham, Nicholas J. (2002). *Accuracy and Stability of Numerical Algorithms*. Second. Society for Industrial and Applied Mathematics. DOI: 10.1137/1.9780898718027. URL: <https://epubs.siam.org/doi/abs/10.1137/1.9780898718027>.